LAB 04

Function-Level Performance Analysis Using cProfile and pstats

Finding where a program spends its time

System Configuration and Performance Evaluation Lab
Course No: 0613-310

Ekramul Haque Amin
Lecturer, AI and Data Science
Dhaka International University (DIU)

From Lab 03 to Lab 04

Labs 01–03

We timed whole programs: one CPU time and one wall time for each run.

Today we ask

Inside a program, which function uses the most time, and why?

To answer this we use a profiler

Today’s Objectives

1. Understand the difference between timing and profiling
2. Profile a Python program with cProfile and pstats
3. Read the report: number of calls, total time and cumulative time
4. Explain primitive vs total calls in recursive functions
5. Use the profile to find the hotspot worth optimizing

Deliverable: Lab Report 04

Timing vs Profiling

Timing

- One number for the whole program
- Tells you how long
- Labs 01–03

```
Total: 0.180 s
```

Profiling

- Time and call count for every function
- Tells you where and why
- Today

```
fibonacci     0.118 s
calculate_sum 0.041 s
…
```

Timing, Profiling and Tracing

Technique	What it records	Python example
Timing	Start and end time of the whole job	time.perf_counter()
Profiling ← today	Time and number of calls for each function	cProfile + pstats
Tracing	Every event in order, as it happens	sys.settrace, trace module

More detail = more information, but also more overhead

Why Profile? Amdahl's Law

```
Overall speedup = 1 / ( (1 - p) + p / s )
```

p = share of time in the part you improve · s = how much faster it gets

If we improve…	p	s	Whole program
fibonacci, 10× faster	0.66	10	2.5× faster
calculate_multiplication, infinitely fast	0.11	∞	1.1× faster

Speed up the biggest part first: the profiler shows which

Two Modules, Two Jobs

STEP 1 · COLLECT

`cProfile`

- Watches the program while it runs
- Records every call and return
- Counts calls and adds up time per function

STEP 2 · REPORT

`pstats`

- Reads what cProfile collected
- Sorts it (by time, calls, name)
- Filters and prints the report

Both come with Python. Nothing to install.

How to Profile: The Pattern

```
import cProfile
import pstats

profiler = cProfile.Profile()
profiler.enable()                # 1. start recording
main()
profiler.disable()               # 2. stop recording

stats = pstats.Stats(profiler)   # 3. load the results
print_report(stats)              # 4. print total_time and cumulative_time
```

The Lab Program

Required by the sheet	Our function	Work it does
Summation	calculate_sum(n)	1 + 2 + … + 1,000,000
Repeated multiplication	calculate_multiplication(n)	1 × 2 × … × 10,000
Recursive function	recursive_factorial(n)	500! (factorial)
Function that calls functions	generate_report()	Calls sum and multiplication, then a short task
Fibonacci series	fibonacci_series(count), fibonacci(n)	First 25 Fibonacci numbers, recursively
main()	main()	Calls everything and prints the results

Code: Sum, Product and Factorial

```
def calculate_sum(n):
    total = 0
    for number in range(1, n + 1):
        total = total + number
    return total

def calculate_multiplication(n):
    result = 1
    for number in range(1, n + 1):
        result = result * number
    return result
```

```
def recursive_factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * recursive_factorial(n - 1)
```

recursive_factorial(500) calls itself 499 more times: watch for 500/1 in the report

Code: Calling Functions and Fibonacci

```
def generate_report():
    sum_result = calculate_sum(1_000_000)        # calls a function
    product = calculate_multiplication(10_000)   # calls another
    size_bits = product.bit_length()             # short extra task
    return sum_result, size_bits

def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)  # two calls each time

def fibonacci_series(count):
    return [fibonacci(i) for i in range(count)]
```

Code: Profiling and Our Report

```
def print_report(stats):
    for (file, line, name), data in stats.stats.items():
        (primitive_calls, total_calls,
         total_time, cumulative_time, callers) = data
        if name in OUR_FUNCTIONS:
            print(total_calls, primitive_calls,
                  total_time, cumulative_time, name)

profiler = cProfile.Profile()
profiler.enable()
main()
profiler.disable()
print_report(pstats.Stats(profiler))
```

We print the numbers in stats.stats with clear names

The Profiling Report

```
PROFILING REPORT (only our functions)
393322 function calls (39 primitive calls) in 0.180 seconds
Ordered by: cumulative_time

    ncalls  total_time  percall  cumulative_time  percall  function
         1       0.000    0.000            0.180    0.180  main
         1       0.000    0.000            0.118    0.118  fibonacci_series
 392809/25       0.118    0.000            0.118    0.005  fibonacci
         1       0.000    0.000            0.061    0.061  generate_report
         1       0.041    0.041            0.041    0.041  calculate_sum
         1       0.020    0.020            0.020    0.020  calculate_multiplication
     500/1       0.001    0.000            0.001    0.001  recursive_factorial
```

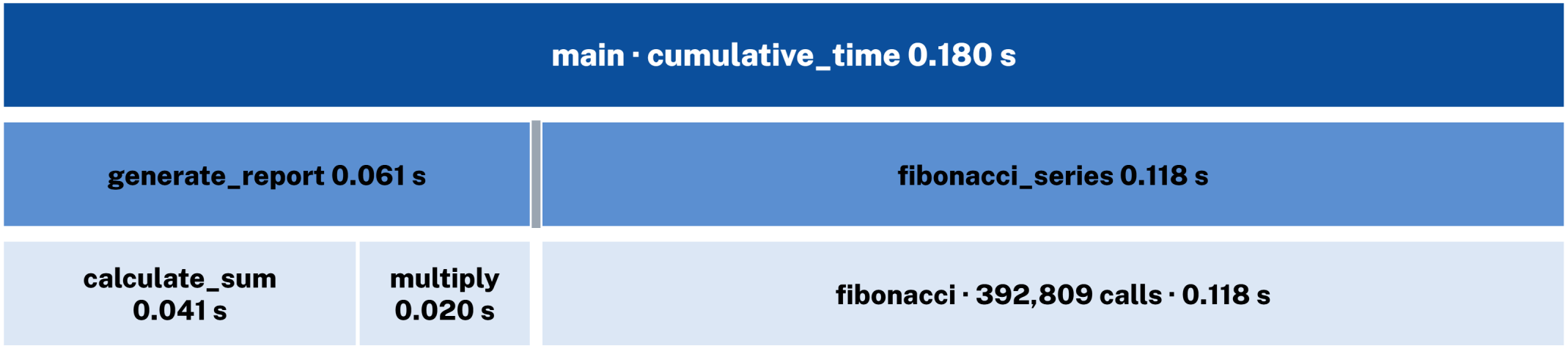
Output from our lab program. Next: what each column means.

What Each Column Means

Column	Meaning
Number of calls	How many times the function was called (total/primitive for recursion)
Total time	Time inside the function's own code, not counting functions it calls
Per call	total_time ÷ ncalls
Cumulative time	Total time, including every function it calls
Per call	cumulative_time ÷ primitive calls
Function	Name of the function

The two most important columns: total_time and cumulative_time

The Report as a Picture



- Width of a bar = cumulative_time: the function plus everything below it
- total_time = the part of a bar with nothing underneath it

Reading the Report

Step 1 / 3 · total_time

Function	ncalls	total_time	cumulative_time
main	1	0.000	0.180
fibonacci_series	1	0.000	0.118
fibonacci	392809/25	0.118	0.118
generate_report	1	0.000	0.061
calculate_sum	1	0.041	0.041
calculate_multiplication	1	0.020	0.020
recursive_factorial	500/1	0.001	0.001

Highest total_time: fibonacci, 0.118 s

total_time counts only the function's own code.

Each fibonacci call is tiny, but it ran 392,809 times.

→ Analysis question 1

Reading the Report

Step 2 / 3 · cumulative_time

Function	ncalls	total_time	cumulative_time
main	1	0.000	0.180
fibonacci_series	1	0.000	0.118
fibonacci	392809/25	0.118	0.118
generate_report	1	0.000	0.061
calculate_sum	1	0.041	0.041
calculate_multiplication	1	0.020	0.020
recursive_factorial	500/1	0.001	0.001

Highest cumulative_time: main, 0.180 s

cumulative_time includes everything a function calls.

generate_report: total_time 0.000, but cumulative_time 0.061 = calculate_sum 0.041 + calculate_multiplication 0.020

→ Analysis question 2

Reading the Report

Step 3 / 3 · ncalls

Function	ncalls	total_time	cumulative_time
main	1	0.000	0.180
fibonacci_series	1	0.000	0.118
fibonacci	392809/25	0.118	0.118
generate_report	1	0.000	0.061
calculate_sum	1	0.041	0.041
calculate_multiplication	1	0.020	0.020
recursive_factorial	500/1	0.001	0.001

total / primitive

392809/25: 392,809 calls in total. Only 25 were primitive (from fibonacci_series); the rest came from fibonacci calling itself.

500/1: called once by main, then 499 times by itself.

→ Analysis questions 3 and 4

Useful pstats Options

Code	What it does
`sort_stats("cumulative")`	Biggest cumulative_time first
`sort_stats("time")`	Biggest total_time first
`sort_stats("calls")`	Most-called first
`print_stats(5)`	Only the top 5 rows
`print_stats("fibonacci")`	Only names that match
`print_callers("fibonacci")`	Who called this function

No code changes: python -m cProfile -s cumulative profiling_lab.py

Fixing the Hotspot

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Version	Calls	Time
Plain recursion	392,809	0.118 s
With lru_cache	25	< 0.001 s

The cache remembers each answer, so each Fibonacci number is computed only once.

Profile → fix the biggest cost → profile again

Lab Task

1. Write a program with the six required parts (slide 9)
2. Run: python profiling_lab.py
3. Check that every printed result is correct
4. Screenshot the profiling report
5. Fill in the results table for every function
6. Answer the four analysis questions with your numbers

> **Extra: sort the rows by total_time and see how the order changes**

Results Table for the Report

Function	Primitive calls	Total calls	total_time (s)	cumulative_time (s)
main				
generate_report				
calculate_sum				
calculate_multiplication				
recursive_factorial				
fibonacci_series				
fibonacci				

Analysis Questions

1. Which function had the highest total time (total_time)? Explain why.
2. Which function had the highest cumulative time? How did the functions it calls add to it?
3. What is the difference between primitive and total call count?
4. Why does a recursive function usually have a larger total call count?

> **Slides 16–18 show how to answer each one**

Common Mistakes to Avoid

Mistake	Fix
Inputs too small (sum to 1,000)	Everything shows 0.000; use bigger inputs
Mixing up total_time and cumulative_time	total_time = own code only; cumulative_time = with calls
Report full of built-in functions	Print only the names in OUR_FUNCTIONS
Recursive fibonacci(35)	About 30 million calls: far too slow when profiled
Comparing with Lab 02–03 times	The profiler adds overhead; profiled code runs slower

Key Takeaway

A profiler shows where the time goes, so you fix the part that matters.

total_time = own code · cumulative_time = with calls · ncalls = total/primitive

> **Next lab: Interpreting measured data: mean, SD, confidence intervals and measurement errors**